kubernetes

k8s

Concepts de Kubernetes

- Kubernetes est un système de gestion de conteneurs
- Il exécute et gère les applications conteneurisées sur un cluster
- Qu'est-ce que cela signifie vraiment?

Choses de base que nous pouvons demander à Kubernetes de faire

- Démarrer 5 conteneurs en utilisant l'image eshop/api:v1.3
- Placer un équilibreur de charge interne devant ces conteneurs
- Démarrer 10 conteneurs en utilisant l'image eshop/webfront:v1.3
- Placer un équilibreur de charge public devant ces conteneurs
- C'est période de soldes, les pics de trafic, développer notre cluster et ajouter des conteneurs
- Nouvelle version! Remplacer mes conteneurs par la nouvelle image eshop/webfront:v1.4
- Continuer à traiter les demandes pendant la mise à niveau; mettre à jour mes conteneurs un par un

Autres choses que Kubernetes peut faire

- Mise à l'échelle automatique de base (autoscaling)
- Déploiement bleu/vert, déploiement Canary
- Services de longue durée, mais aussi travaux par lots (ponctuels)
- Surengager notre cluster et éjecter les tâches à faible priorité
- Exécuter des services avec des données avec état (bases de données, etc.)
- Contrôle d'accès précis définissant ce qui peut être fait par qui sur quelles ressources
- Intégration de services tiers (catalogue de services)
- Automatiser des tâches complexes (opérateurs)

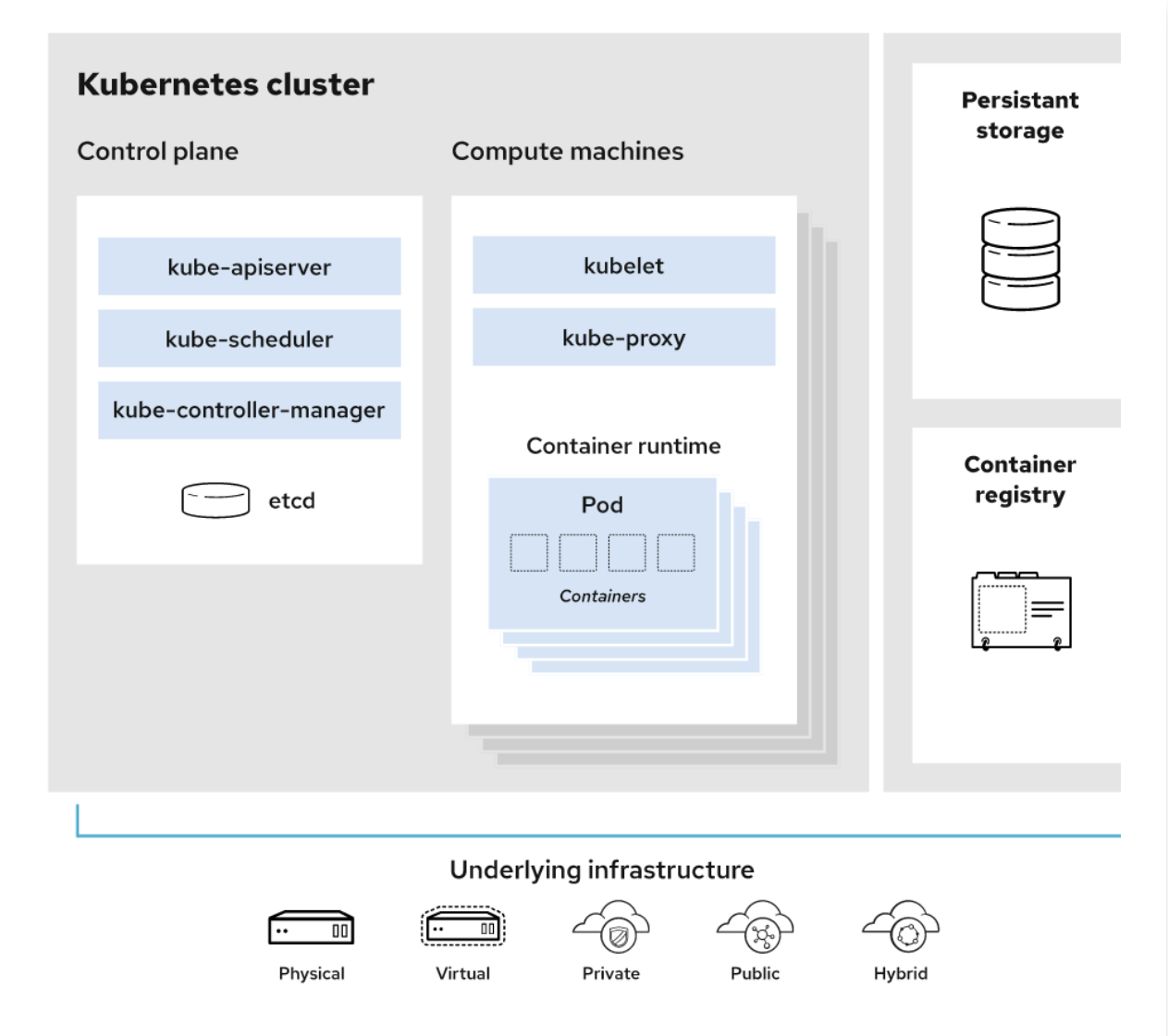
Fonctionnalités

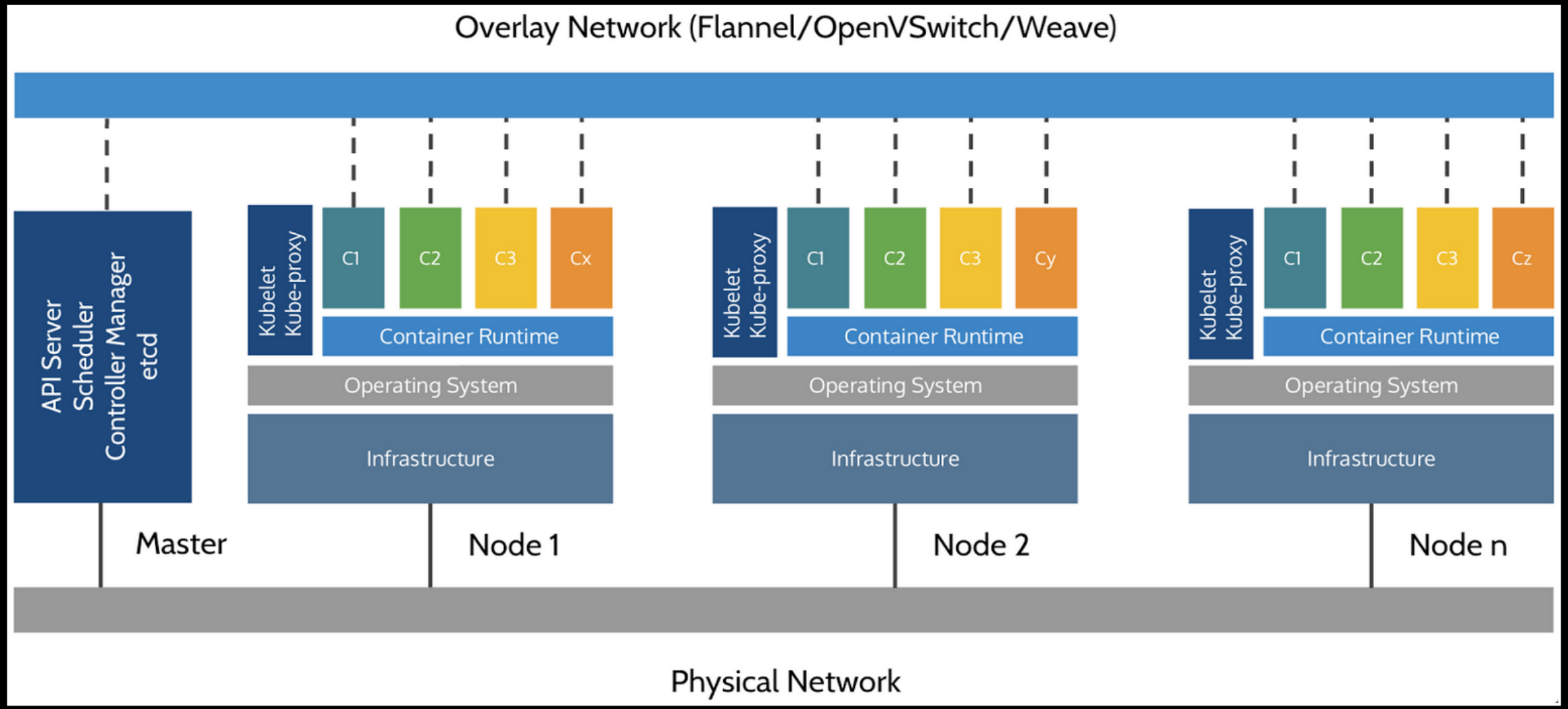
- **Découverte de service et équilibrage de charge** Kubernetes peut exposer un conteneur en utilisant le nom DNS ou en utilisant sa propre adresse IP. Si le trafic vers un conteneur est élevé, Kubernetes est capable d'équilibrer la charge et de distribuer le trafic réseau afin que le déploiement soit stable.
- **Orchestration du stockage** Kubernetes vous permet de monter automatiquement un système de stockage de votre choix, tel que des stockages locaux, des fournisseurs de cloud public, etc.
- **Déploiements et annulations automatisés** Vous pouvez décrire l'état souhaité pour vos conteneurs déployés à l'aide de Kubernetes, et il peut changer l'état réel à l'état souhaité à une vitesse contrôlée. Par exemple, vous pouvez automatiser Kubernetes pour créer de nouveaux conteneurs pour votre déploiement, supprimer des conteneurs existants et adopter toutes leurs ressources dans le nouveau conteneur.

Fonctionnalités

- **Empaquetage** automatique de binaires Vous fournissez à Kubernetes un cluster de nœuds qu'il peut utiliser pour exécuter des tâches en conteneur. Vous indiquez à Kubernetes la quantité de CPU et de mémoire (RAM) dont chaque conteneur a besoin. Kubernetes peut installer des conteneurs sur vos nœuds pour tirer le meilleur parti de vos ressources.
- **L'auto- réparation** Kubernetes redémarre les conteneurs qui échouent, remplace les conteneurs, tue les conteneurs qui ne répondent pas à votre vérification de l'état définie par l'utilisateur et ne les annonce pas aux clients tant qu'ils ne sont pas prêts à être diffusés.
- **Gestion des secrets et de la configuration** Kubernetes vous permet de stocker et de gérer des informations sensibles, telles que les mots de passe, les jetons OAuth et les clés SSH. Vous pouvez déployer et mettre à jour les secrets et la configuration de l'application sans reconstruire vos images de conteneur et sans exposer les secrets dans la configuration de votre pile.

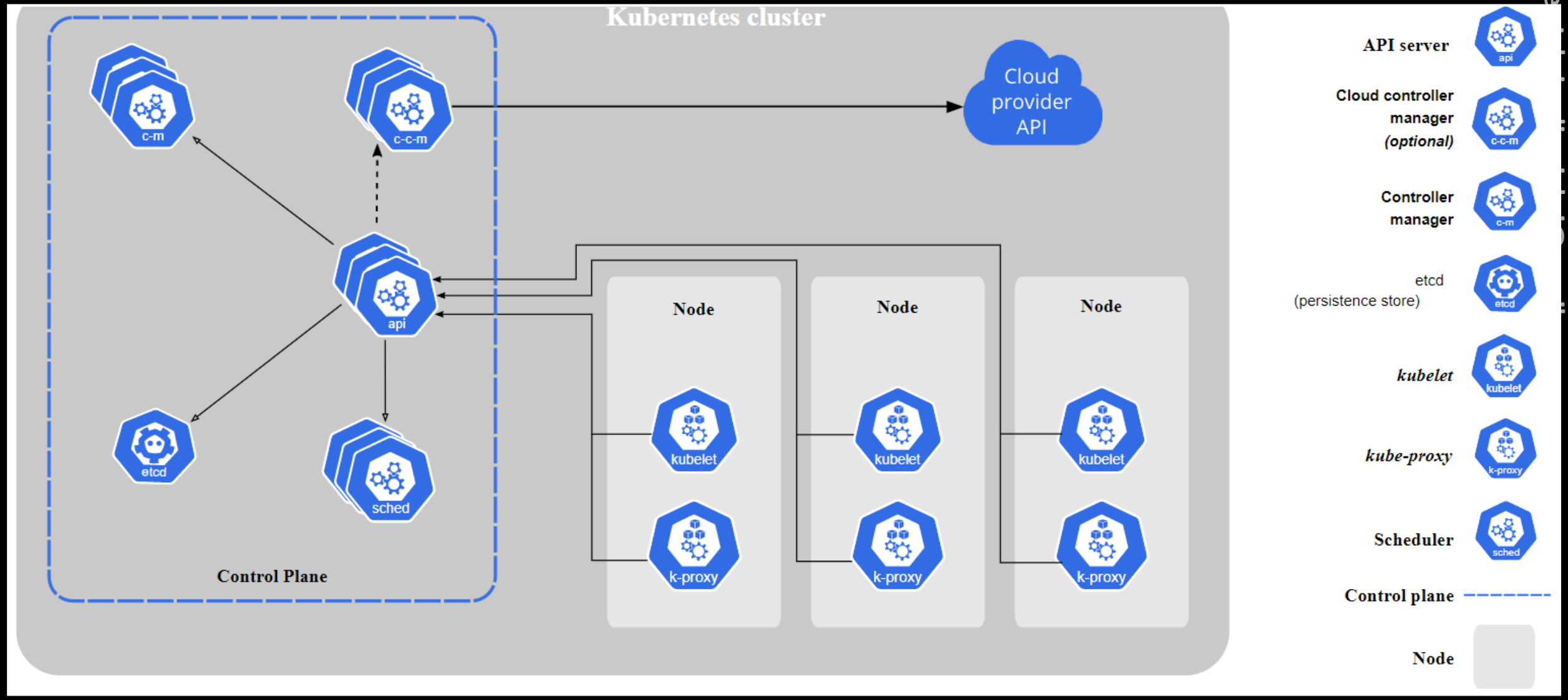
Kubernetes Cluster





Architecture de Kubernetes

Elies Jebri : elies.jebri@gmail.com



Composants de Kubernetes

Composants du plan de contrôle

- Les composants du plan de contrôle prennent des décisions globales concernant le cluster (par exemple, la planification), ainsi que la détection et la réponse aux événements du cluster (par exemple, le démarrage d'un nouveau pod lorsque le champ *replicas* d'un déploiement n'est pas satisfait).
- Les composants du plan de contrôle peuvent être exécutés sur n'importe quelle machine du cluster.
- Pour plus de simplicité, les scripts de configuration démarrent généralement tous les composants du plan de contrôle sur la même machine et n'exécutent pas de conteneurs utilisateur sur cette machine.
- **Le plan de contrôle est également appelé le "master"**

kube-apiserver

- Le serveur API est un composant du plan de contrôle de Kubernetes qui expose l'API Kubernetes.
- Le serveur API est le frontal du plan de contrôle Kubernetes.
- La principale implémentation d'un serveur d'API Kubernetes est kube-apiserver .
- kube-apiserver est conçu pour évoluer horizontalement, c'est-à-dire qu'il évolue en déployant plus d'instances.
- Vous pouvez exécuter plusieurs instances de kube-apiserver et équilibrer le trafic entre ces instances.

etcd

Magasin de valeurs de clé cohérent et hautement disponible utilisé comme magasin de sauvegarde de Kubernetes pour toutes les données du cluster.

Si votre cluster Kubernetes utilise etcd comme magasin de sauvegarde, assurez-vous d'avoir un plan de sauvegarde pour ces données.

Elies Jebri : elies.jebri@gmail.com

kube-scheduler

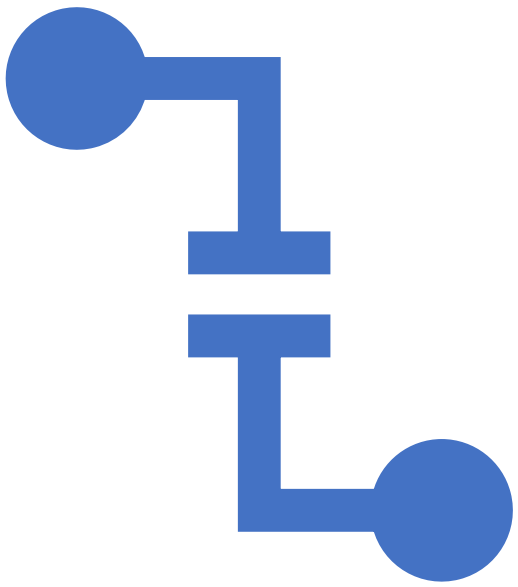
- Composant du plan de contrôle qui surveille les nouveaux Pods sans nœud assigné et sélectionne un nœud sur lequel les exécuter.
- Les facteurs pris en compte pour les décisions de planification comprennent: les besoins en ressources individuelles et collectives, les contraintes matérielles / logicielles / politiques, les spécifications d'affinité et d'anti-affinité, la localisation des données, l'interférence entre les charges de travail et les délais.

kube-controller-manager

- Composants du Plan de Controle qui exécutent les processus de controle.
- Logiquement, chacun controleur est un processus distinct, mais pour réduire la complexité, ils sont tous compilés en un seul binaire et exécutés dans un seul processus.
- Ces contrôleurs comprennent:
  - Contrôleur de nœud: responsable de remarquer et de répondre lorsque les nœuds tombent en panne.
  - Contrôleur de réplication: responsable de la gestion du nombre correct de pods pour chaque objet contrôleur de réplication dans le système.
  - Contrôleur Endpoints: remplit l'objet Endpoints (c'est-à-dire joint les services et les pods).
  - Contrôleurs de compte de service et de jetons: créez des comptes par défaut et des jetons d'accès API pour les nouveaux espaces de noms.

cloud-controller-manager

- Un plan de contrôle Kubernetes qui intègre une logique de contrôle spécifique au cloud. Le gestionnaire de contrôleur cloud vous permet de lier votre cluster à l'API de votre fournisseur cloud et sépare les composants qui interagissent avec cette plate-forme cloud des composants qui interagissent simplement avec votre cluster.
- Le cloud-controller-manager n'exécute que les contrôleurs spécifiques à votre fournisseur de cloud. Si vous exécutez Kubernetes dans vos propres locaux ou dans un environnement d'apprentissage à l'intérieur de votre propre PC, le cluster n'a pas de gestionnaire de contrôleur cloud.
- Comme avec le kube-controller-manager, le cloud-controller-manager combine plusieurs boucles de contrôle logiquement indépendantes en un seul binaire que vous exécutez en tant que processus unique. Vous pouvez mettre à l'échelle horizontalement (exécuter plusieurs copies) pour améliorer les performances ou pour aider à tolérer les échecs.



cloud-controller-manager

- Les contrôleurs suivants peuvent avoir des dépendances de fournisseur de cloud:
  - Contrôleur de nœud: pour vérifier le fournisseur de cloud afin de déterminer si un nœud a été supprimé dans le cloud après avoir cessé de répondre
  - Contrôleur de route: pour configurer des routes dans l'infrastructure cloud sous-jacente
  - Contrôleur de service: pour créer, mettre à jour et supprimer des équilibreurs de charge de fournisseur de cloud

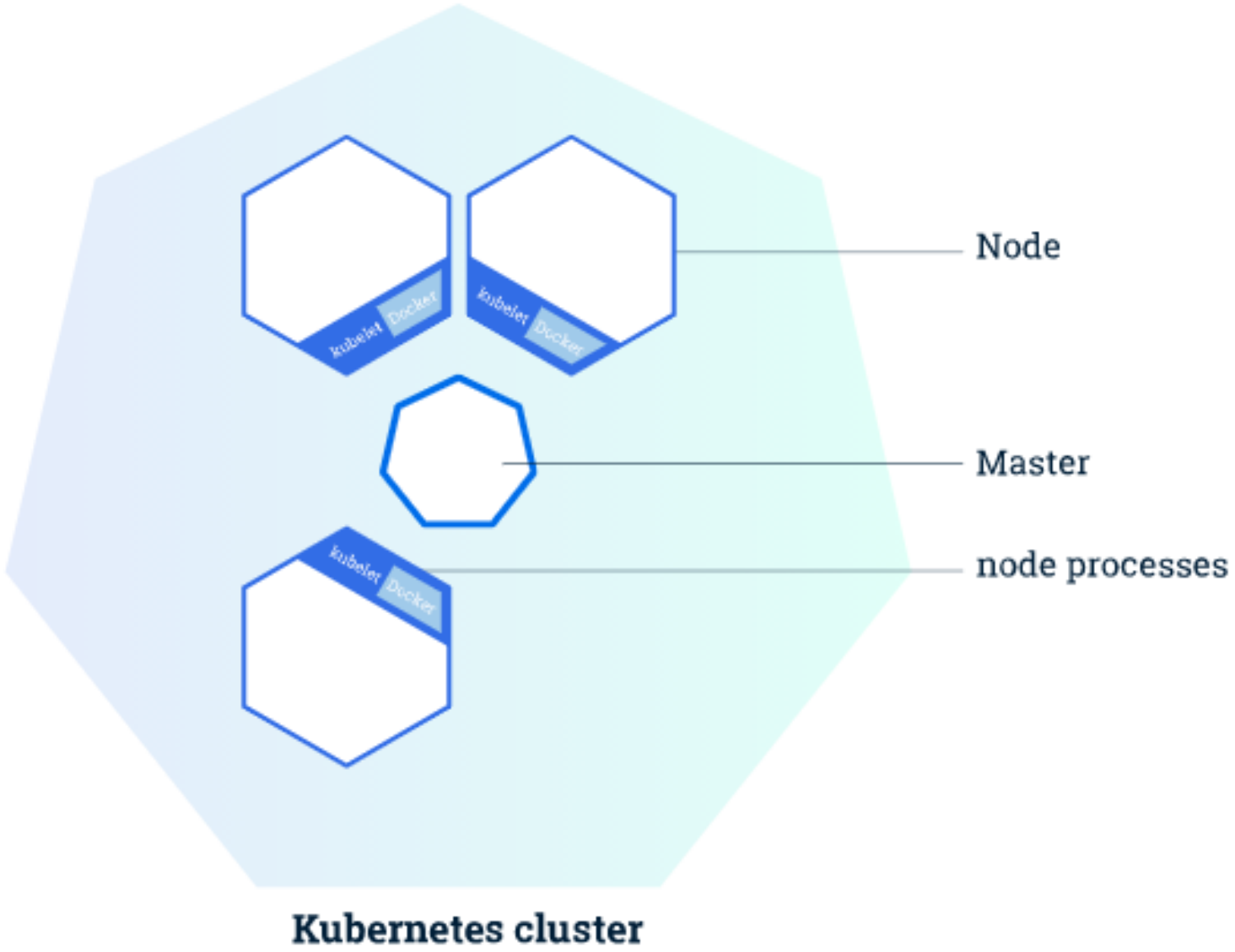
Elies Jebri : elies.jebri@gmail.com

Les nœuds

- Les nœuds exécutant nos conteneurs exécutent une collection de services:
  - un moteur de conteneur (généralement Docker)
  - kubelet (l’agent du nœud)
  - kube-proxy (un composant réseau)
- Les nœuds étaient anciennement appelés "minions"

Composants du nœud

- **kubelet**
  - Un agent qui s'exécute sur chacun nœud dans le cluster. Il s'assure que les conteneurs s’exécutent dans un Pod.
  - Le kubelet prend un ensemble de PodSpecs qui sont fournis via divers mécanismes et garantit que les conteneurs décrits dans ces PodSpecs sont en cours d'exécution et sains. Le kubelet ne gère pas les conteneurs qui n'ont pas été créés par Kubernetes.
- **kube-proxy**
  - kube-proxy est un proxy réseau qui s'exécute sur chaque nœud dans votre cluster.
  - kube-proxy maintient les règles de réseau sur les nœuds. Ces règles réseau permettent la communication réseau avec vos pods à partir de sessions réseau à l'intérieur ou à l'extérieur de votre cluster.
  - kube-proxy utilise la couche de filtrage de paquets du système d'exploitation s'il en existe une et qu'elle est disponible. Sinon, kube-proxy transfère le trafic lui-même.
- **Container runtime**



Cluster Kubernetes

L’API et les Objets Kubernetes

Interagir avec Kubernetes

- L'API Kubernetes est RESTful
- Elle permet de CRUD des ressources ou objets
- Types courants de ressources :
  - **node** (Bare-metal ou VM - dans le cluster)
  - **pod** (groupe de conteneurs fonctionnant ensemble sur un nœud)
  - **service** (point de terminaison réseau stable pour se connecter à un ou plusieurs conteneurs)
- Déclarer les objets pour décrire l'état souhaité d'un cluster :
  - quelles applications ou autres processus exécuter,
  - quelles images utiliser,
  - nombre de replicas,
  - les ressources réseau et disque à disposer, etc

Interaction avec K8s

- On définit des objets via l'interface en ligne de commande **kubectl** de deux façons :
  - en lançant une commande *kubectl run <conteneur> ..., kubectl expose ...*
  - en décrivant un objet dans un fichier *YAML* ou *JSON* et en le passant au client

    kubectl apply -f monpod.yml
- Vous pouvez utiliser directement l'API Kubernetes pour interagir avec le cluster et définir ou modifier l'état souhaité. Kubernetes est complètement **automatisable** !

Parenthèse : Le YAML

- Kubernetes décrit ses ressources en YAML.

```yaml
- marché:
    lieu: Marché de la Place
    jour: jeudi
    horaire:
      unité: "heure"
      min: 12
      max: 20
    fruits:
      - nom: pomme
        couleur: "verte"
        pesticide: avec

      - nom: poires
        couleur: jaune
        pesticide: sans
    légumes:
      - courgettes
      - salade
      - potiron
```

Syntaxe

- Alignement ! (2 espaces !!)
- Des paires clé: valeur
- Un peu comme du JSON, avec la différence que le JSON n’impose pas l’alignement et met des accolades et des points-virgules
- Les extensions Kubernetes et YAML dans VSCode vous aident à repérer des erreurs

Syntaxe de base d’une description YAML Kubernetes

- Les description YAML permettent de décrire de façon lisible et manipulable de nombreuses caractéristiques des ressources Kubernetes (un peu comme un Compose file par rapport à la CLI Docker).

```yaml
kind: Service
apiVersion: v1
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard
  namespace: kubernetes-dashboard
spec:
  ports:
    - port: 443
      targetPort: 8443
  selector:
    k8s-app: kubernetes-dashboard
  type: NodePort
```

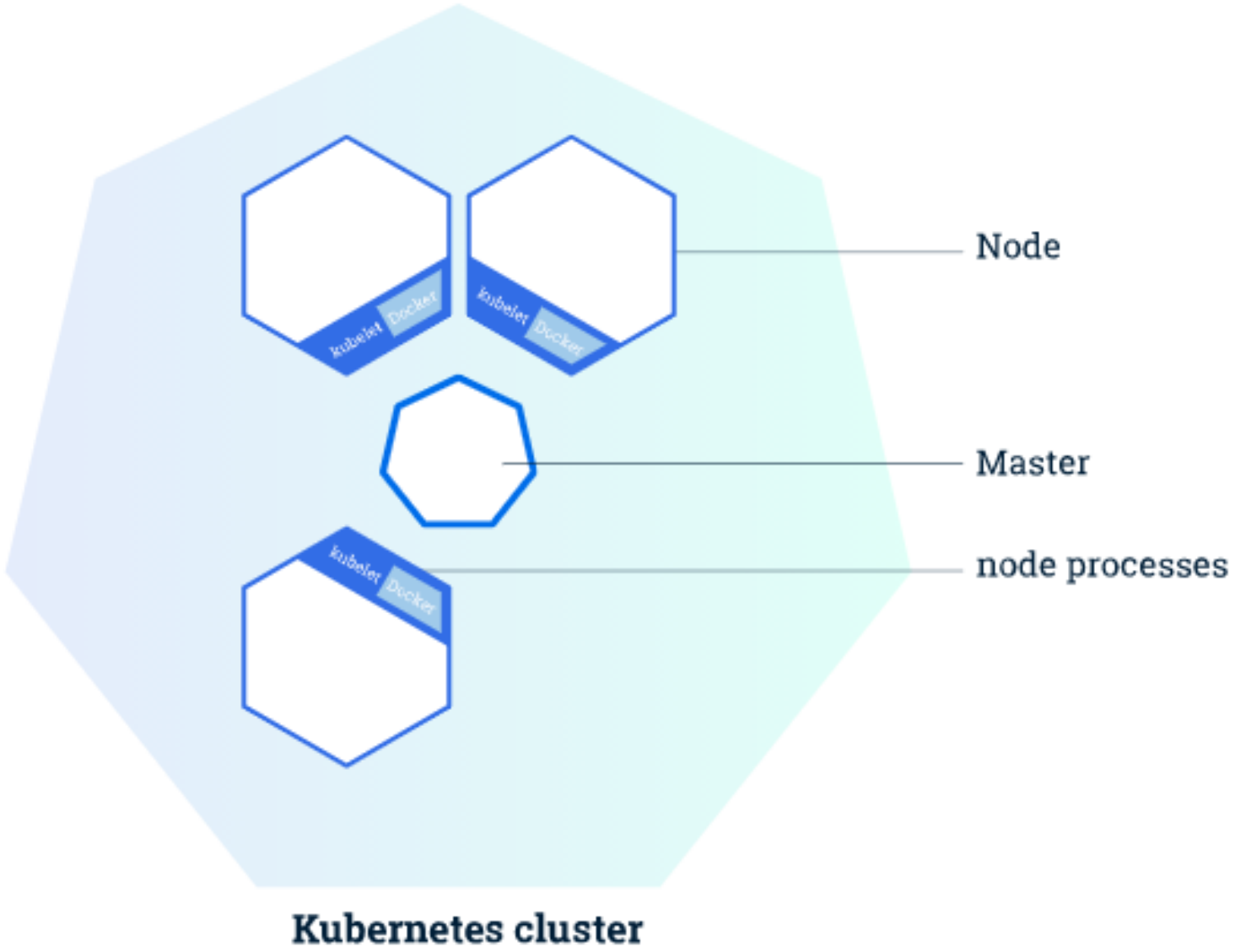
OBJETS DE BASE

Les nœuds

- Les nœuds exécutant nos conteneurs exécutent une collection de services:
  - un moteur de conteneur (généralement Docker)
  - kubelet (l’agent du nœud)
  - kube-proxy (un composant réseau)
- Les nœuds étaient anciennement appelés "minions"

Composants du nœud

- **kubelet**
  - Un agent qui s'exécute sur chacun nœud dans le cluster. Il s'assure que les conteneurs s’exécutent dans un Pod.
  - Le kubelet prend un ensemble de PodSpecs qui sont fournis via divers mécanismes et garantit que les conteneurs décrits dans ces PodSpecs sont en cours d'exécution et sains. Le kubelet ne gère pas les conteneurs qui n'ont pas été créés par Kubernetes.
- **kube-proxy**
  - kube-proxy est un proxy réseau qui s'exécute sur chaque nœud dans votre cluster.
  - kube-proxy maintient les règles de réseau sur les nœuds. Ces règles réseau permettent la communication réseau avec vos pods à partir de sessions réseau à l'intérieur ou à l'extérieur de votre cluster.
  - kube-proxy utilise la couche de filtrage de paquets du système d'exploitation s'il en existe une et qu'elle est disponible. Sinon, kube-proxy transfère le trafic lui-même.
- **Container runtime**



Cluster Kubernetes

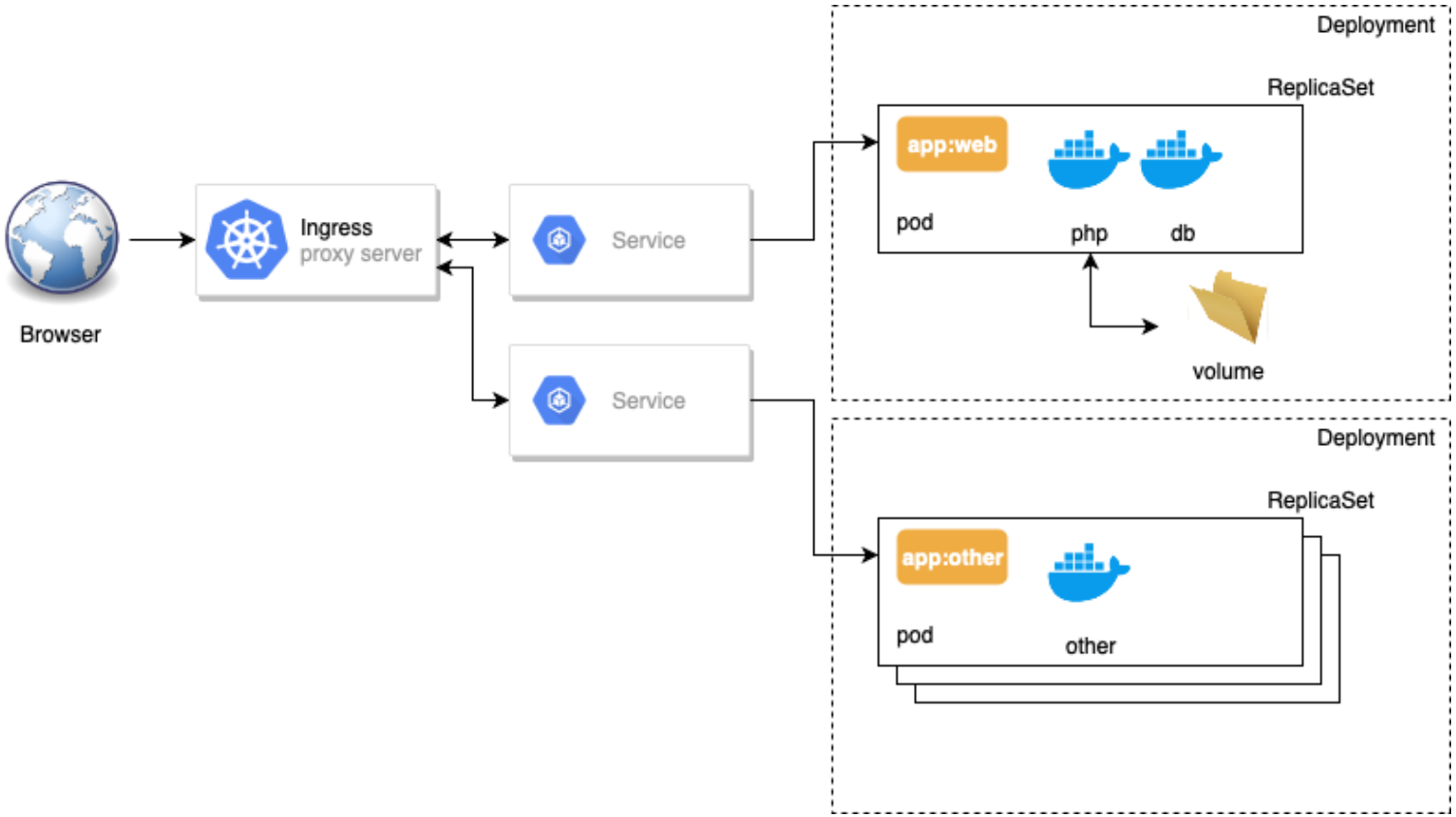
Node

Un nœud est une machine (Vm ou machine physique) qui possède un environnement Container Runtimes.

Cela signifie qu'un pod fonctionne sur un nœud mais peut facilement être dupliqué sur un autre nœud (en cas de pannes par exemple).

L'architecture découplée des services Kubernetes

- L'ensemble de la gestion d'un service applicatif se décompose dans Kubernetes en 3 à 4 objets articulés entre eux:
- replicatset
- deployment
- service
- (ingress)

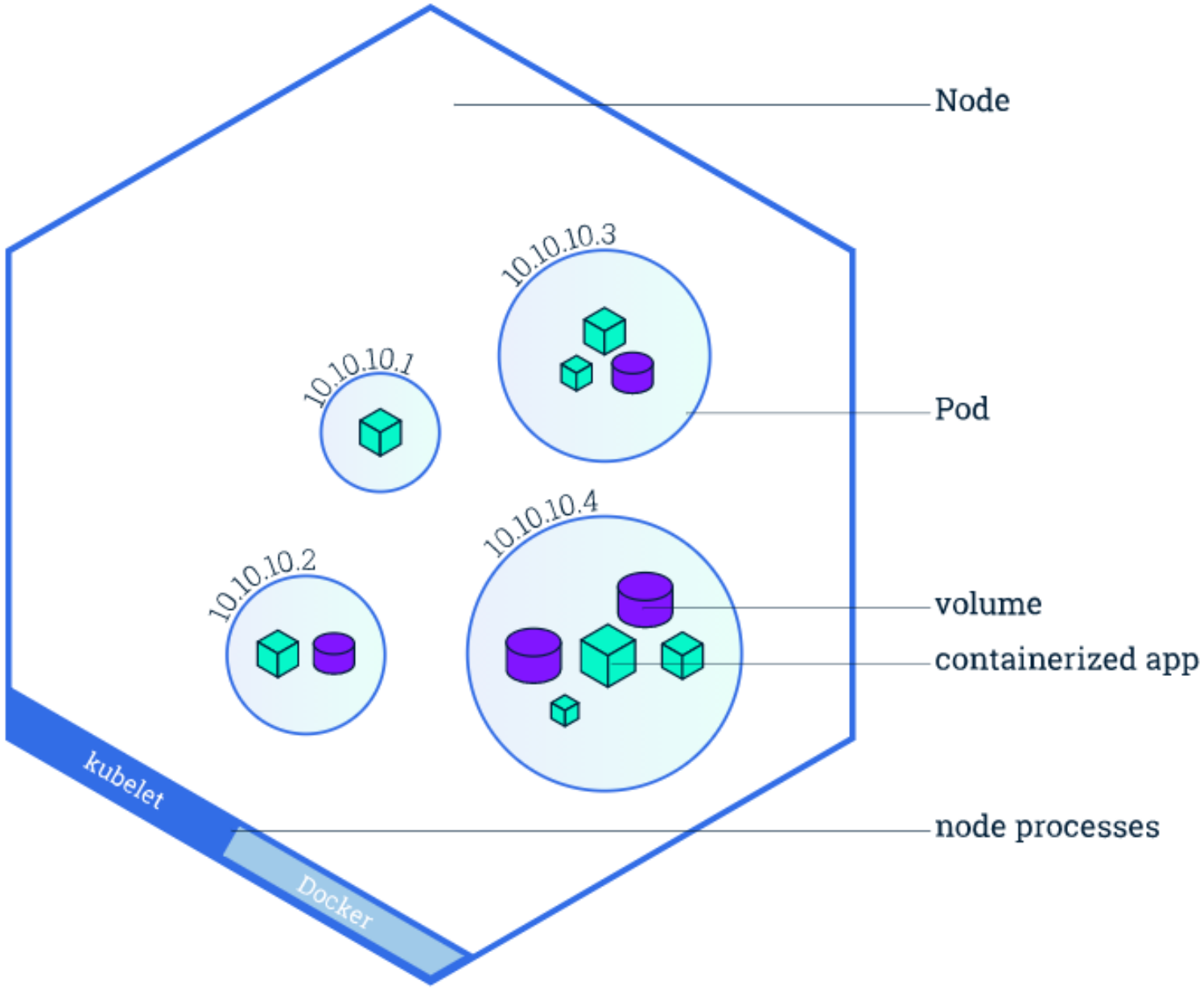


Les namespaces

- Tous les objets Kubernetes sont rangés dans différents espaces de travail isolés appelés namespaces.
- Cette isolation permet 3 choses :
  - ne voir que ce qui concerne une tâche particulière lorsqu’on opère sur un cluster
  - créer des limites de ressources (CPU, RAM, etc.) pour le namespace
  - définir des rôles et permissions sur le namespace qui s’appliquent à toutes les ressources à l’intérieur.

Les Pods

- Un Pod représente une unité de déploiement : un petit nombre de conteneurs qui sont étroitement liés et qui partagent :
  - les mêmes ressources de calcul
  - des volumes communs
  - la même IP donc le même nom de domaine
  - peuvent se parler sur localhost
  - peuvent se parler en IPC
  - ont un nom différent et des logs différents

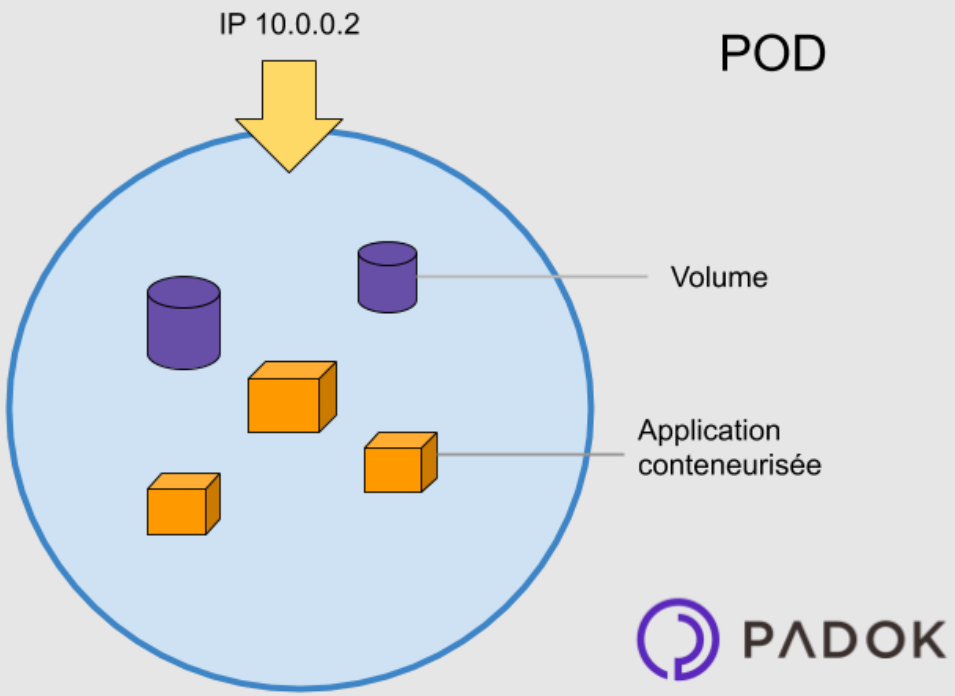


Pods

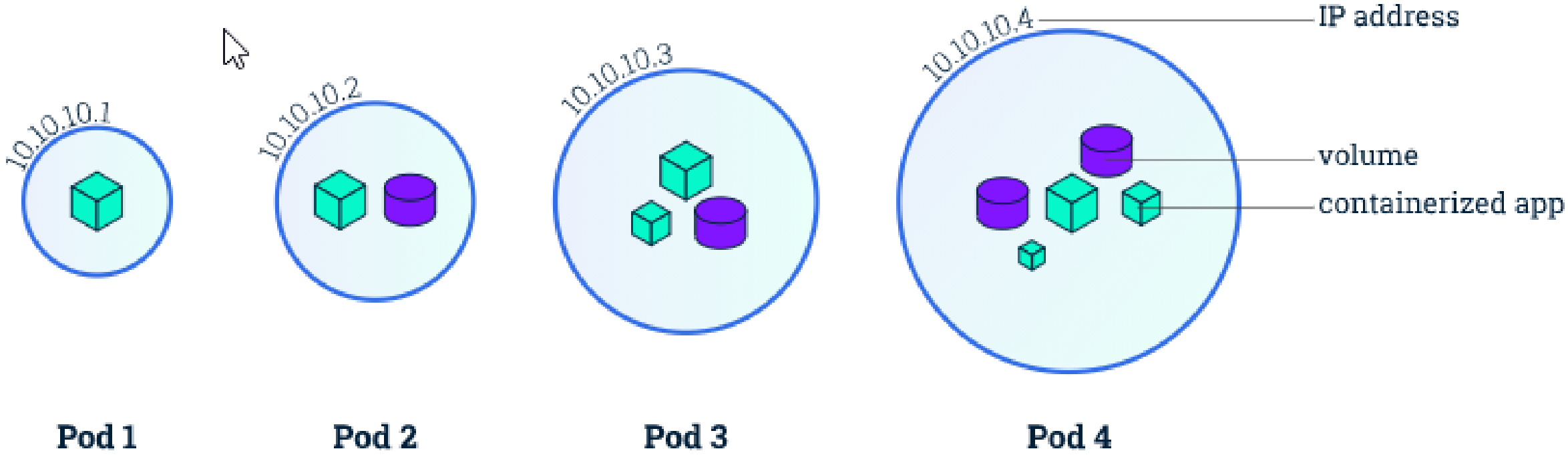
Elies Jebri : elies.jebri@gmail.com

Pod

- Un pod est l'objet de base de Kubernetes. Il englobe les conteneurs, les ressources de stockage et les IP réseau.
- Un pod représente un élément d'une application dans Kubernetes. Ces pods sont lancés dans un cluster Kubernetes qui est composé de noeuds.
- Les pods sont destinés à être éphémères. Kubernetes peut adapter le nombre de ces pods pour s'adapter au trafic entrant, créant ou supprimant ainsi des pods à la demande.
- Kubernetes gère le trafic vers ce nombre fluctuant de pods en utilisant des **services**.



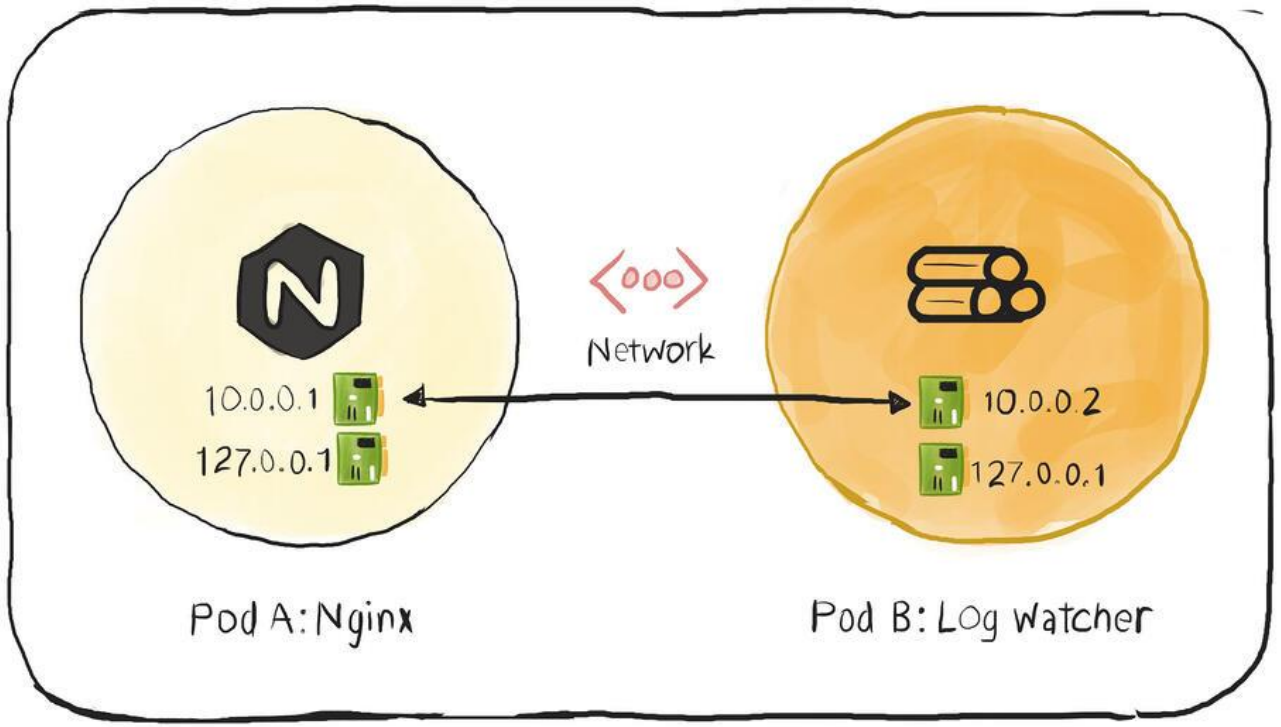
PADOK


10.10.10.1
10.10.10.2
10.10.10.3
10.10.10.4
IP address
volume
containerized app
Pod 1
Pod 2
Pod 3
Pod 4

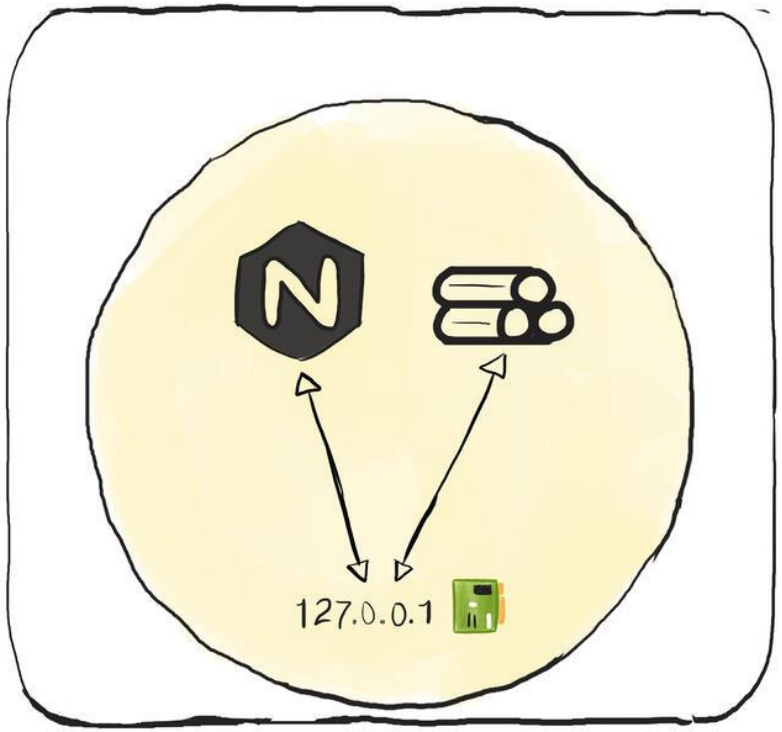
PODs

Containers vs Pods

- Containers



- Pod

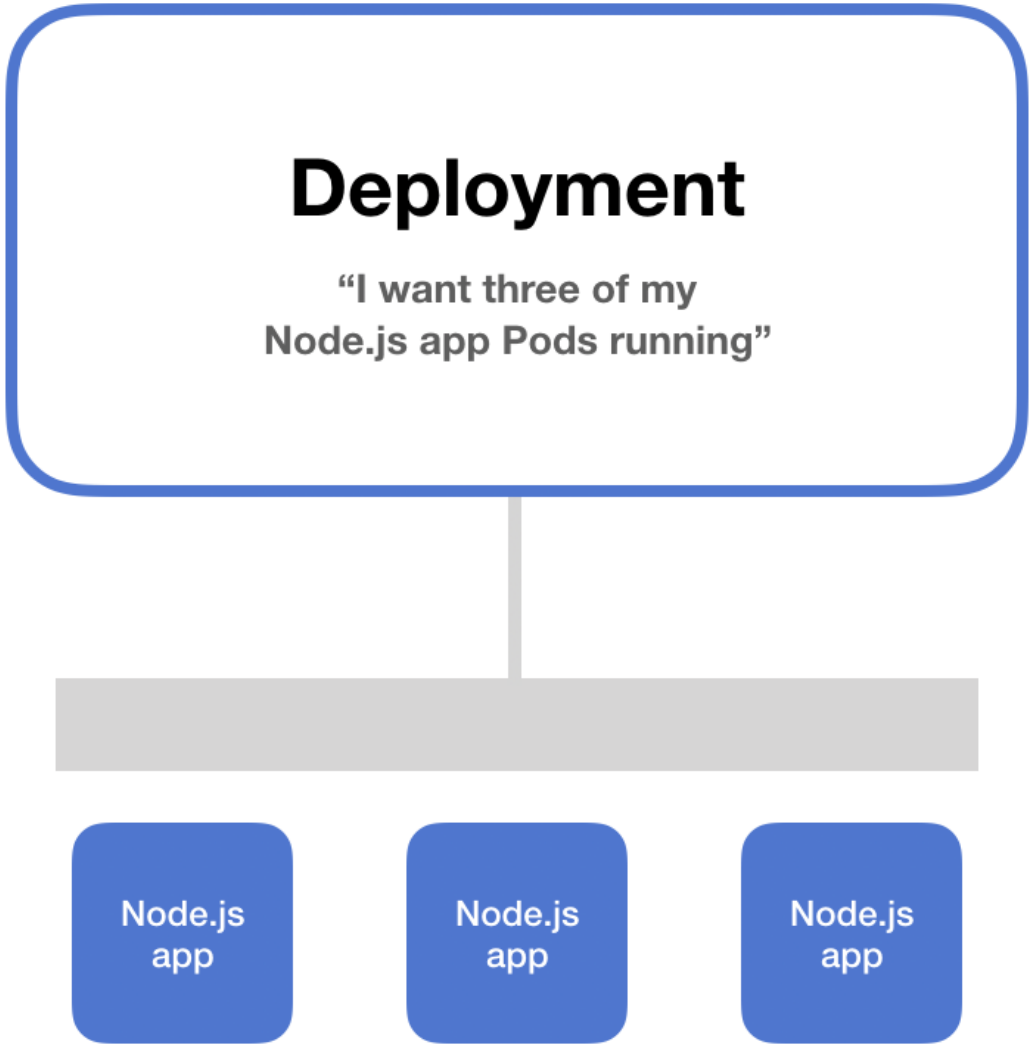


Elies Jebri : elies.jebri@gmail.com

Un manifeste de Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: rancher-demo-pod
spec:
  containers:
    - image: monachus/rancher-demo:latest
      name: rancher-demo-container
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP
    - image: redis
      name: redis-container
      ports:
        - containerPort: 6379
          name: http
          protocol: TCP
```

Déploiements



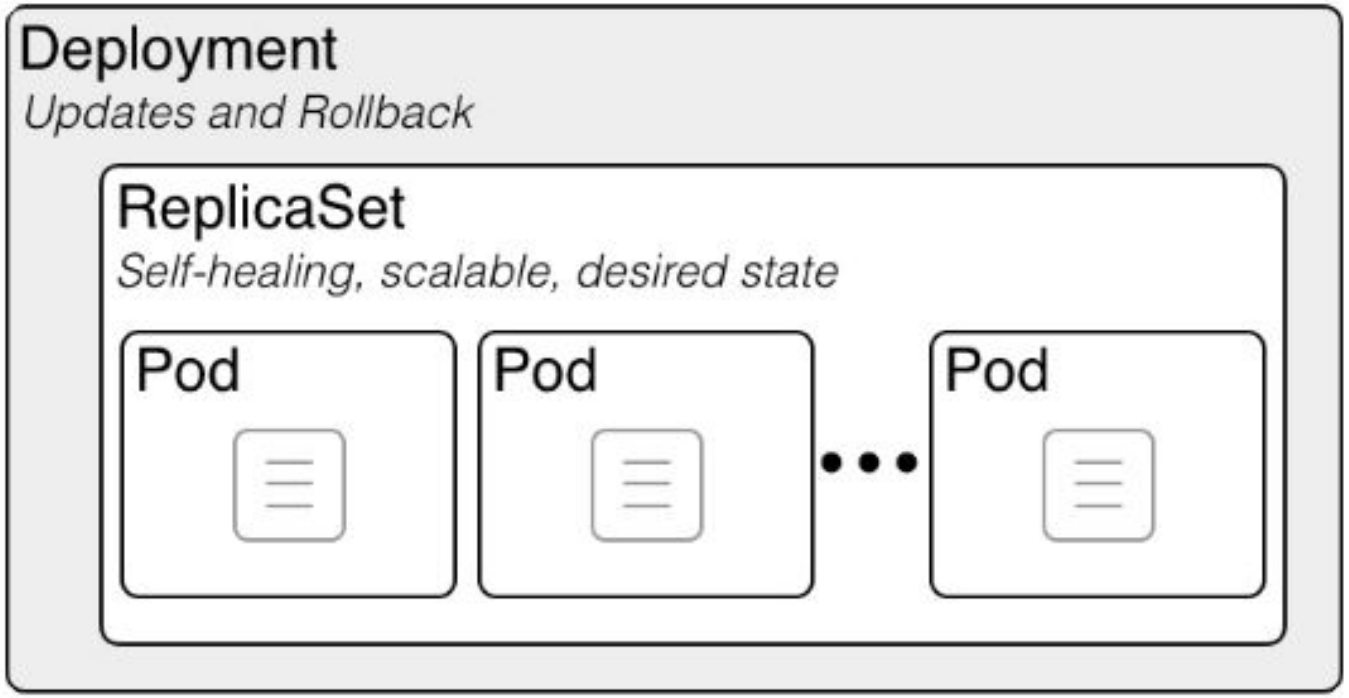
Elies Jebri : elies.jebri@gmail.com

Déploiements

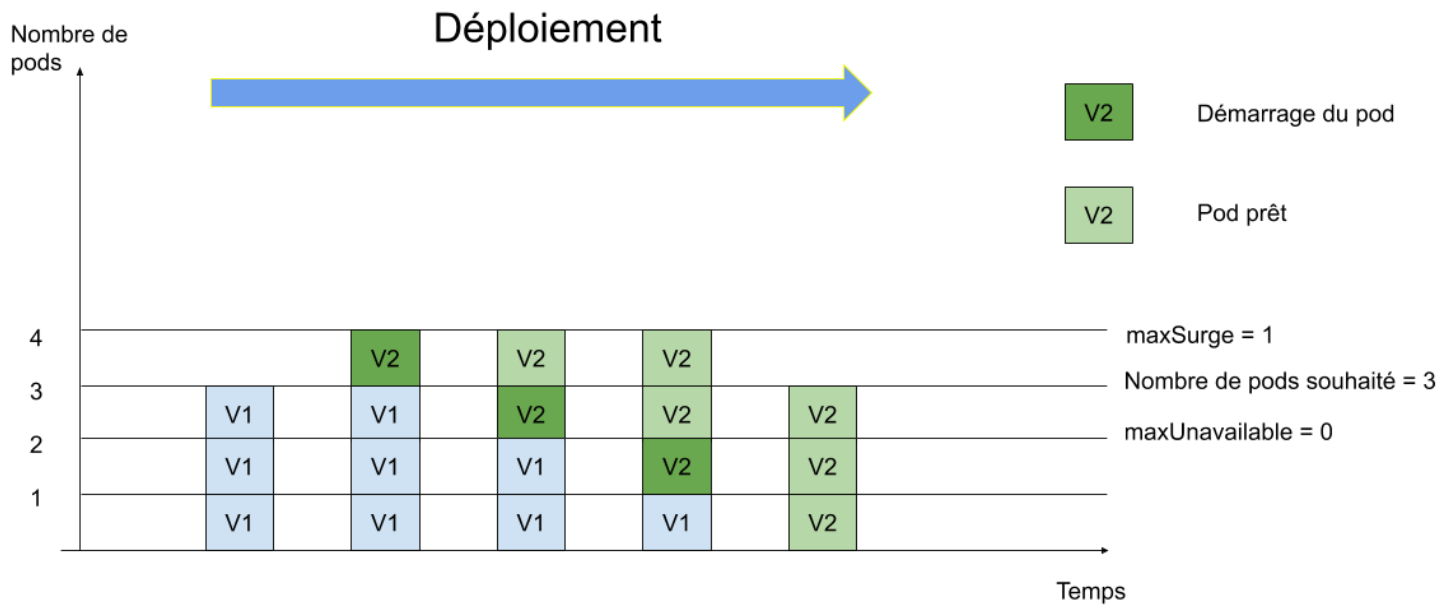
- Objets Kubernetes qui gèrent les pods.
- Permettent de déployer une version spécifique de votre application et de spécifier le nombre de pods dont vous avez besoin pour qu'elle soit opérationnelle.
- Lorsqu'une nouvelle version est prête à être mise en production, le déploiement peut facilement gérer cette mise à jour sans downtime en appliquant deux règles de base :
  - **maxSurge** spécifie le nombre maximum de pods qui peuvent être créés au-delà du nombre de pods désiré.
  - **Max Unavailable** spécifie le nombre maximum de pods qui peuvent être indisponibles pendant le déploiement.

Les Deployments (deploy)

- Ce sont des objets de plus haut niveau que les pods et replicaset.
- Un deployment implique la création d'un ensemble de Pods désignés par une étiquette **label** et regroupés dans un **Replicaset**.



Déploiement



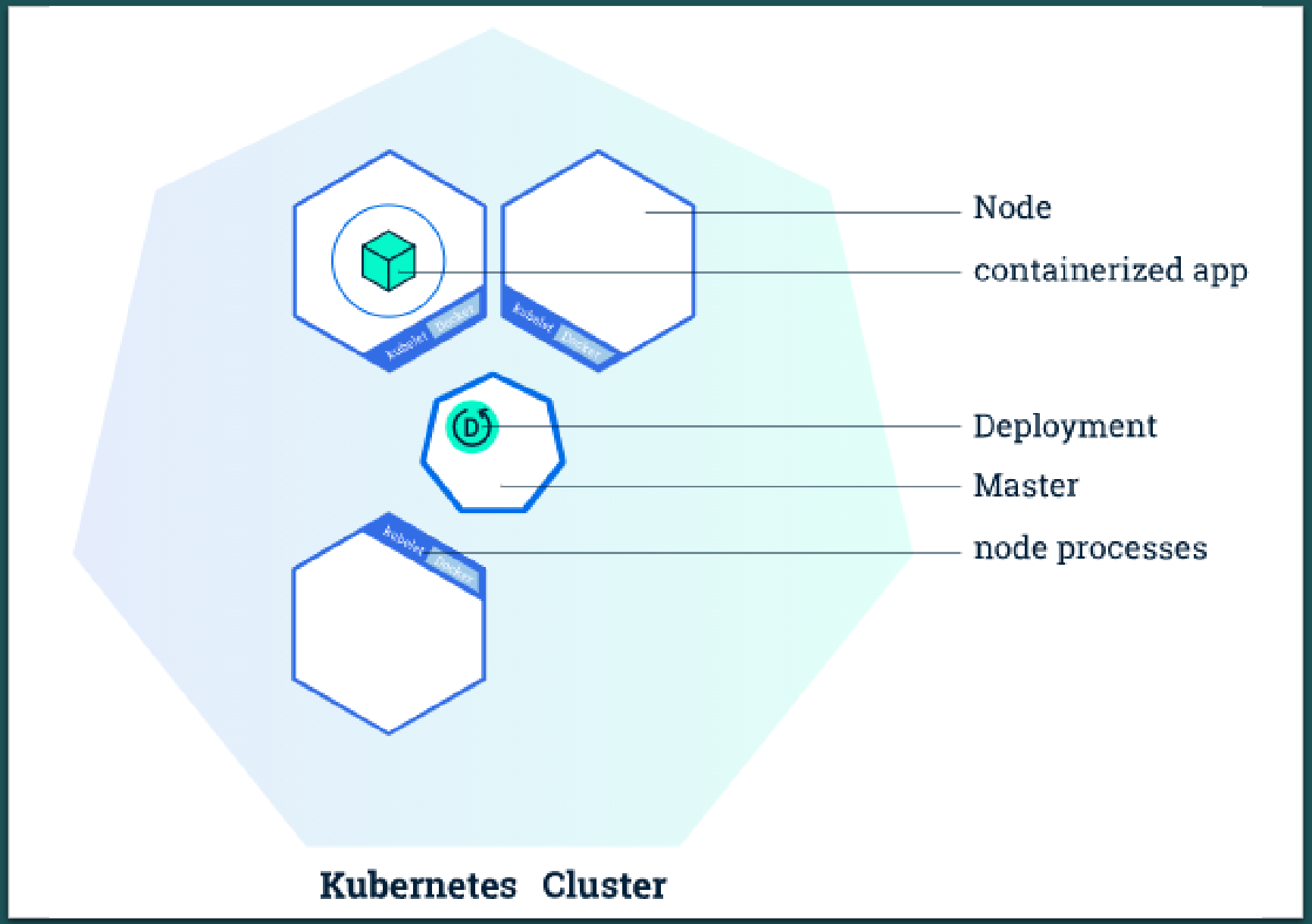
PADOK

- Ces pods sont orchestrés sur différents nœuds du cluster et peuvent être déplacés de l'un à l'autre.
- Avec une telle facilité de déploiement et de mise à niveau des applications, vous ne devez tout même jamais oublier de tester votre application dans un environnement de staging.
- Vous voulez également que tous vos environnements soient indépendants, Kubernetes utilise des configmaps et des secrets pour cela.

Elies Jebri : elies.jebri@gmail.com

Exemple :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.7.9
          ports:
            - containerPort: 80
```



Déploiement d'une application sur Kubernetes

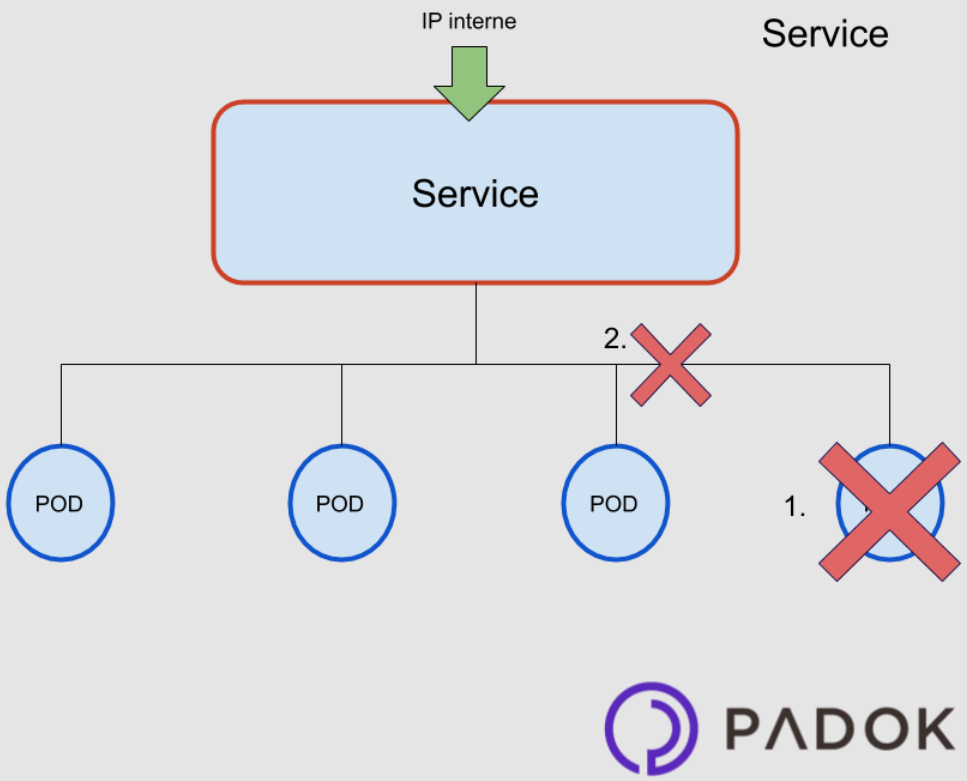
Les ReplicaSets (rs)

- Les ReplicaSet servent à gérer et sont responsables pour:
  - la réplication (avoir le bon nombre d’instances et le scaling)
  - la santé et le redémarrage automatique des pods de l’application (Self-Healing)
  - kubectl get rs pour afficher la liste des replicas.

Les Services

- Désignent un ensemble de pods (grâce à des tags/labels) généralement géré par un déploiement.
- Fournissent un endpoint réseau pour les requêtes à destination de ces pods.
- Configurent une politique permettant d’y accéder depuis l’intérieur ou l’extérieur du cluster.

Service



- La nature éphémère des pods les rend peu fiables pour assurer un trafic fluide. La solution de Kubernetes pour pallier cela est le **Service**.
- Ce service maintient une liste logique de pods qui acceptent le trafic entrant et exposent un port interne pour accéder aux pods sous-jacents.
- Le service Kubernetes peut donc gérer les changements de trafic en modifiant le nombre de pods.

Elies Jebri : elies.jebri@gmail.com

Types de services Kubernetes

ClusterIP . Expose un service qui n'est accessible qu'à partir du cluster.

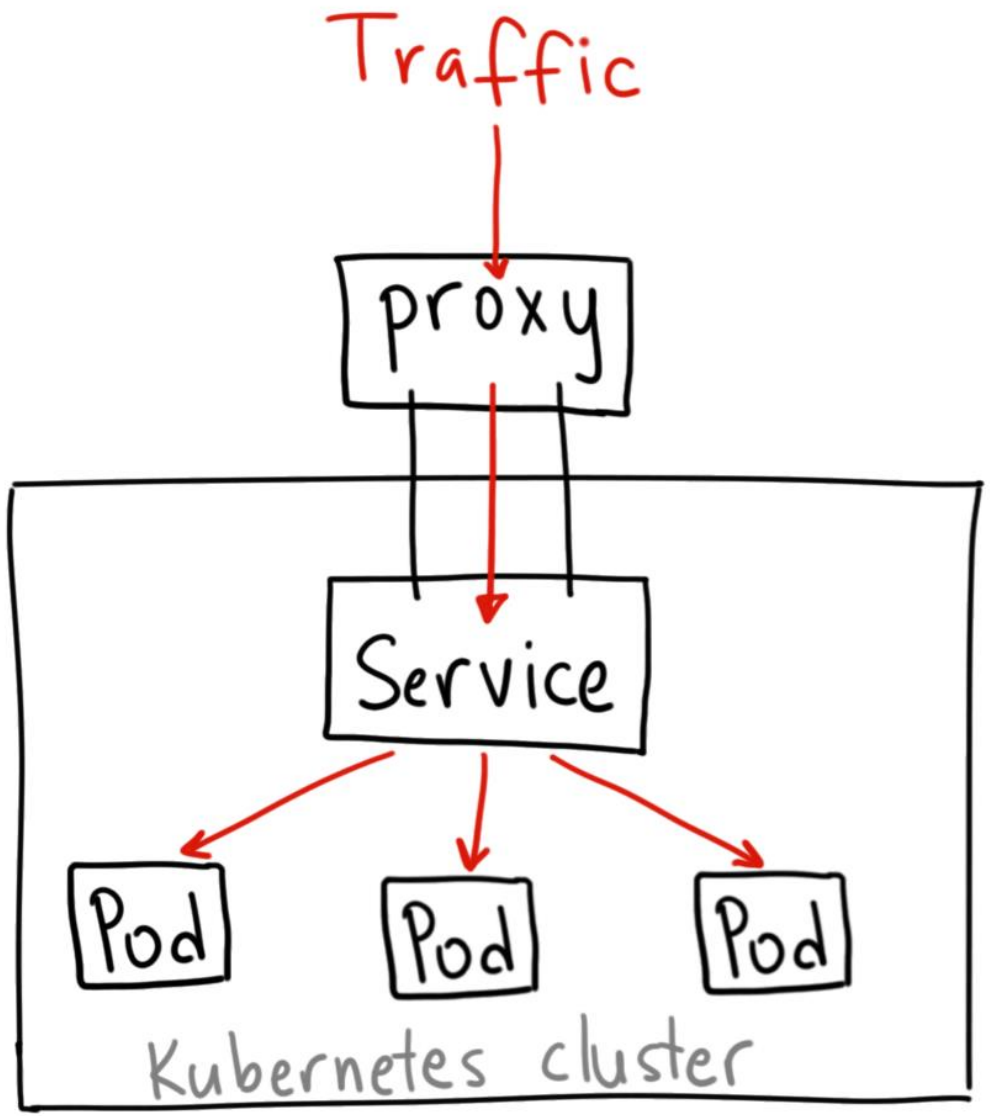
NodePort . Expose un service via un port statique sur l'IP de chaque nœud.

LoadBalancer. Expose le service via l'équilibreur de charge du fournisseur de cloud.

ExternalName . Mappe un service à un champ externalName prédéfini en renvoyant une valeur pour l'enregistrement CNAME.

ClusterIP

- Expose le service sur une IP interne au cluster.
- Les autres pods peuvent alors accéder au service de l'intérieur du cluster, mais il n'est pas exposé à l'extérieur.
- Il faut un proxy pour le connecter à l'extérieur

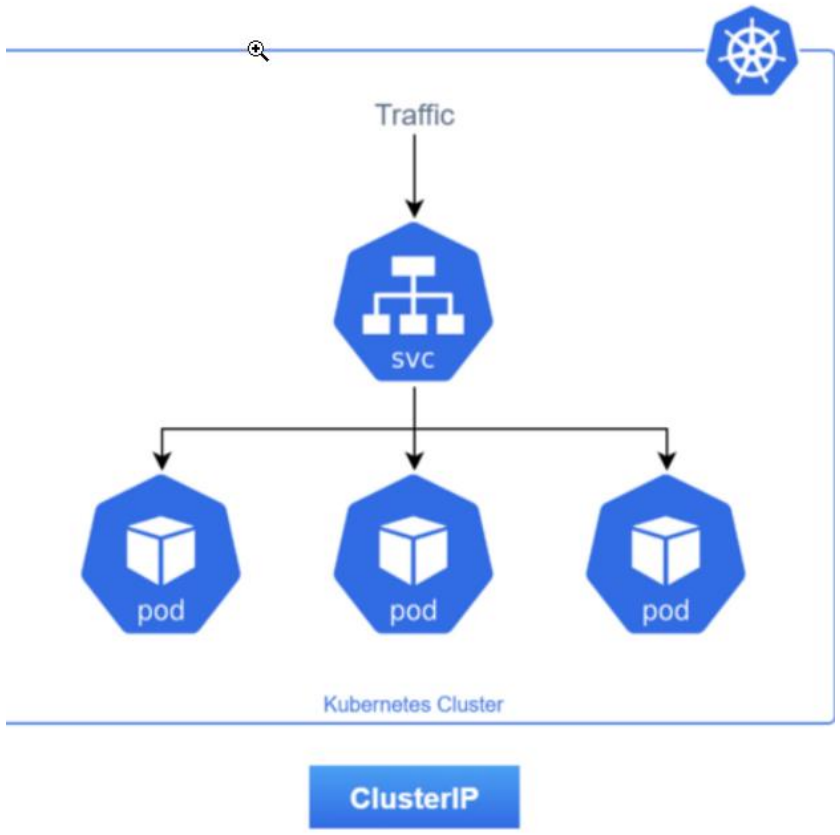


ClusterIP

- ClusterIP est le type de service par défaut et le plus courant.
- Kubernetes attribuera une adresse IP interne au cluster au service ClusterIP. Cela rend le service accessible uniquement au sein du cluster.
- Vous ne pouvez pas faire de demandes de service (pods) depuis l'extérieur du cluster.

- **Cas d'utilisation**

- Communication interservices au sein du cluster. Par exemple, la communication entre les composants front-end et back-end de votre application.

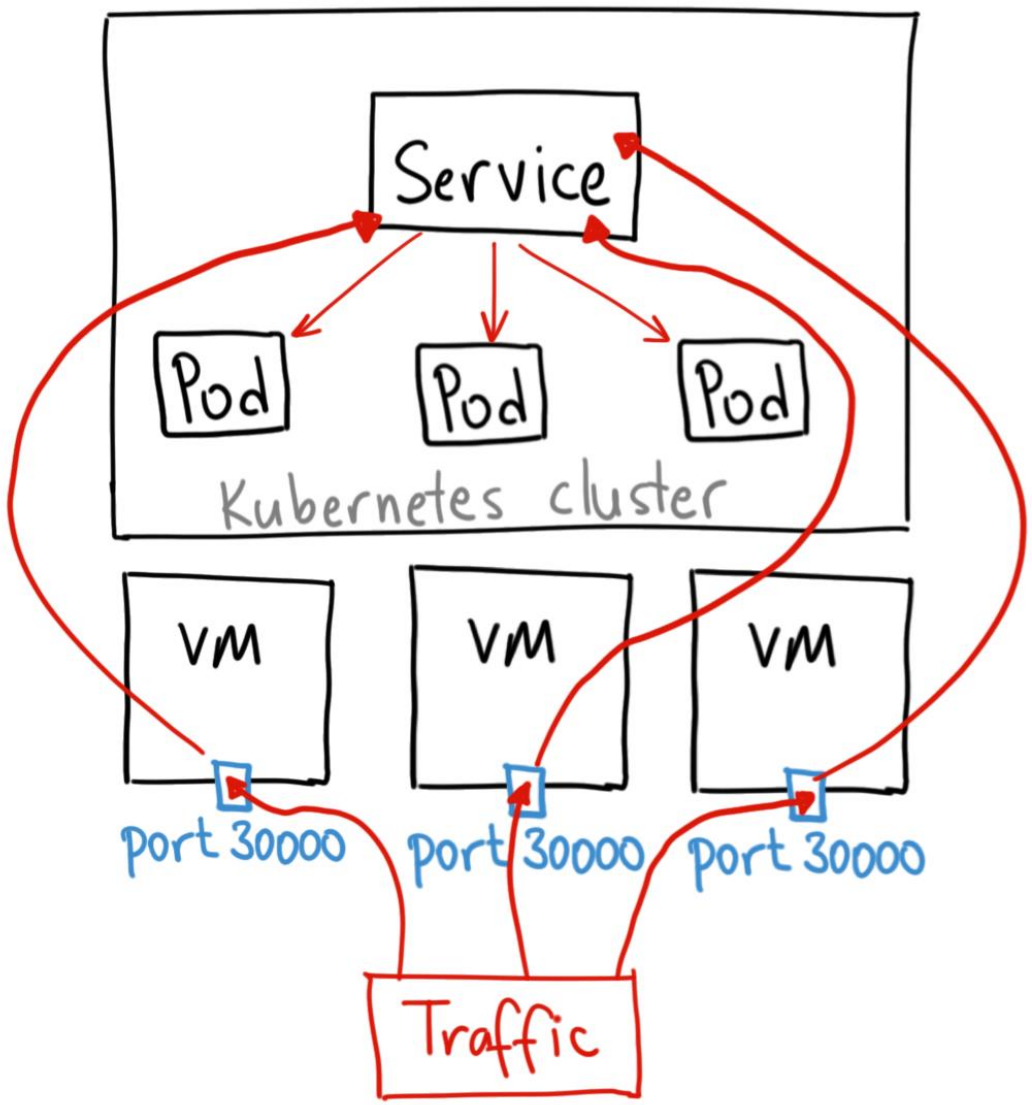


NodePort

expose le service depuis l'IP de chacun des nœuds du cluster en ouvrant un port directement sur le nœud, entre 30000 et 32767.

Cela permet d'accéder aux pods internes répliqués.

Comme l'IP est stable on peut faire pointer un DNS ou Loadbalancer classique dessus.

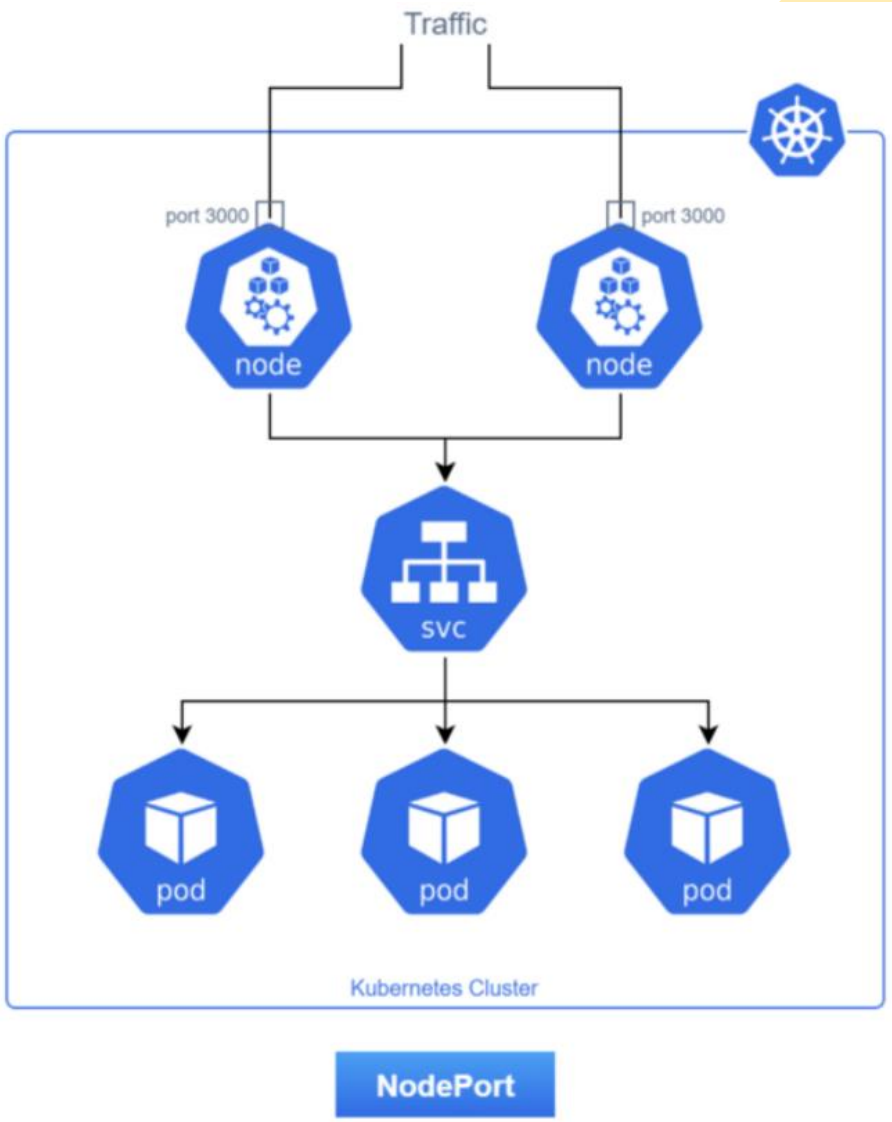


NodePort

- Extension du service ClusterIP vers lequel le service NodePort achemine et qui sera automatiquement créé.
- Expose le service à l'extérieur du cluster et ajoute un port à l'échelle du cluster au-dessus de ClusterIP.
- Expose le service sur l'adresse IP de chaque nœud sur un port statique (le NodePort). Chaque nœud va mandater (proxy) ce port vers votre service. Ainsi, le trafic externe a accès au port fixe sur chaque nœud. Cela signifie que toute demande adressée à votre cluster sur ce port est transmise au service.
- Vous pouvez contacter le service NodePort, depuis l'extérieur du cluster, en demandant **<NodeIP>:<NodePort>.**
- Le port de nœud doit être compris entre 30000 et 32767. L'attribution manuelle d'un port au service est facultative. S'il n'est pas défini, Kubernetes en attribuera automatiquement un.
- Si vous choisissez explicitement le port de nœud, assurez-vous que le port n'a pas déjà été utilisé par un autre service.

Cas d'utilisation

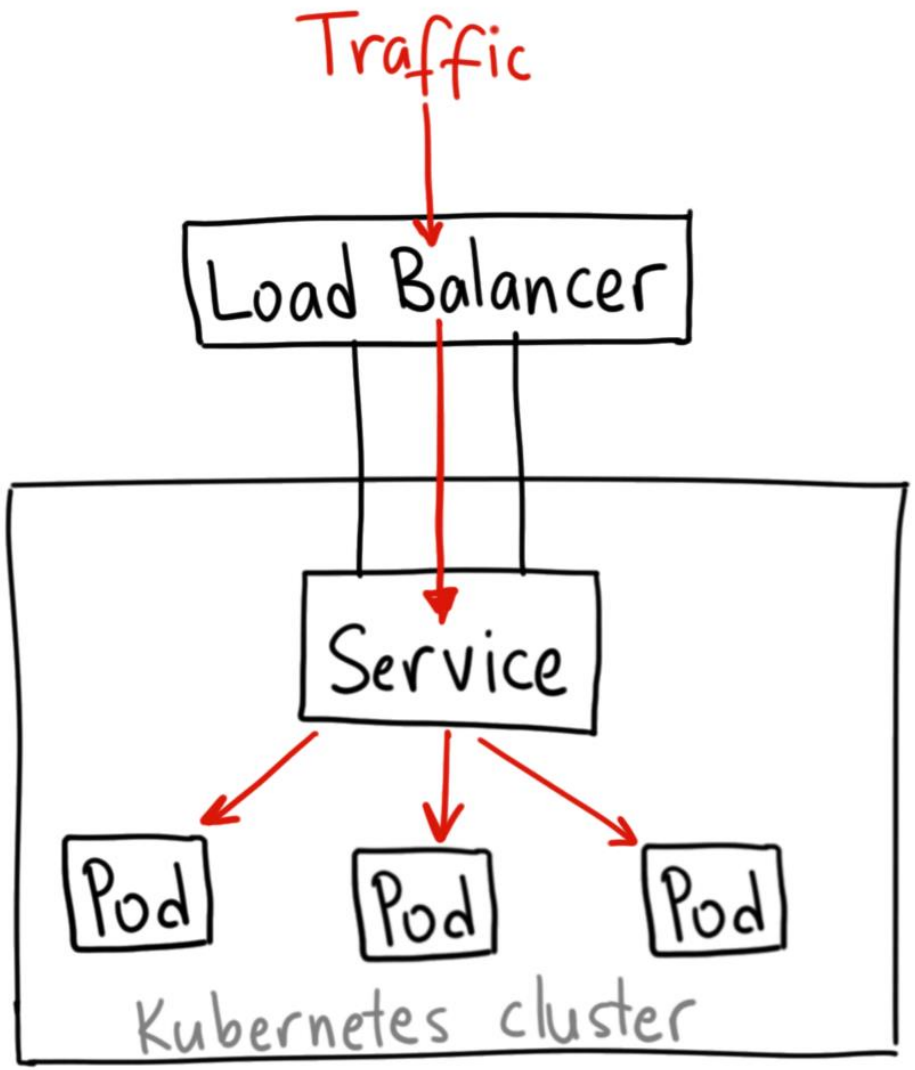
- Lorsque vous souhaitez activer la connectivité externe à votre service.
- L'utilisation d'un NodePort vous donne la liberté de configurer votre propre solution d'équilibrage de charge, de configurer des environnements qui ne sont pas entièrement pris en charge par Kubernetes, ou même d'exposer directement les IP d'un ou plusieurs nœuds.
- Préférez placer un équilibreur de charge au-dessus de vos nœuds pour éviter la défaillance des nœuds.



Load Balancer

Expose le service en externe à l'aide d'un Loadbalancer de fournisseur de cloud.

Les services NodePort et ClusterIP, vers lesquels le Loadbalancer est dirigé sont automatiquement créés.

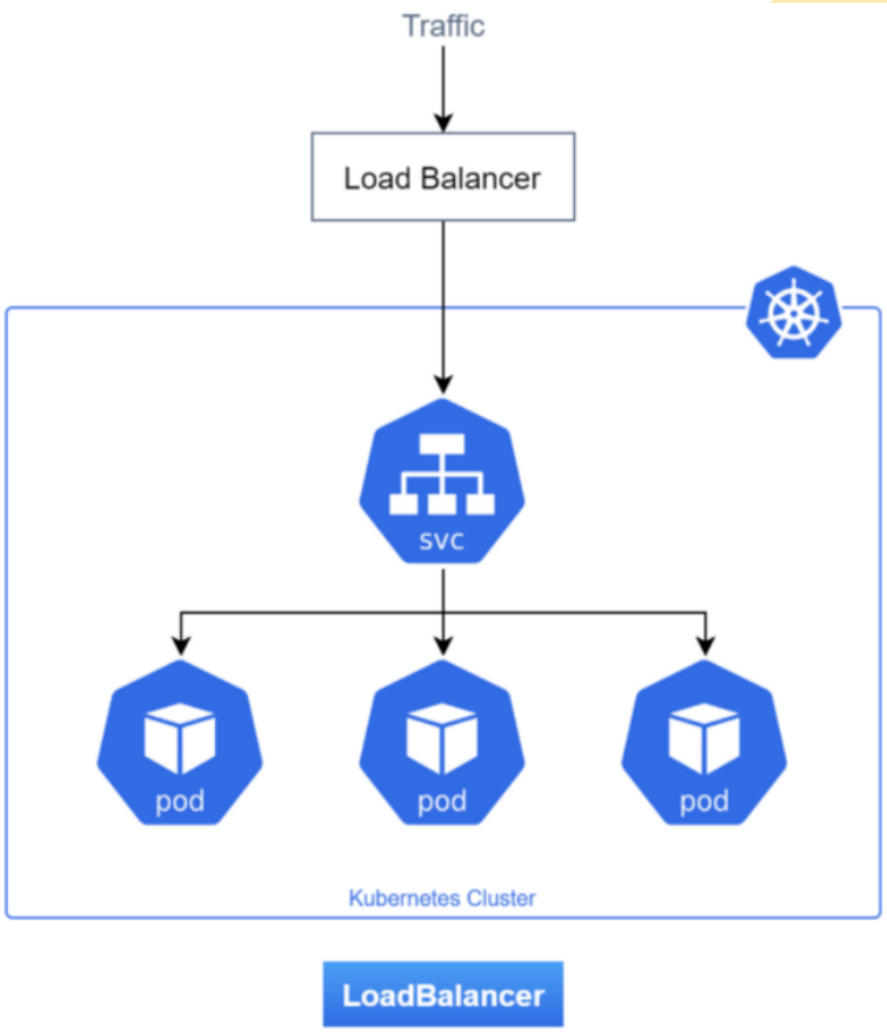


LoadBalancer

- Ce service est une extension du service NodePort. Les services NodePort et ClusterIP, vers lesquels l'équilibreur de charge externe achemine, sont automatiquement créés.
- Il intègre NodePort avec des équilibreurs de charge basés sur le cloud.
- Il expose le Service en externe à l'aide de l'équilibreur de charge d'un fournisseur de cloud.
- Chaque fournisseur de cloud (AWS, Azure, GCP, etc.) possède sa propre implémentation d'équilibreur de charge natif. Le fournisseur de cloud créera un équilibreur de charge, qui acheminera ensuite automatiquement les demandes vers votre service Kubernetes.
- Le trafic provenant de l'équilibreur de charge externe est dirigé vers les pods backend. Le fournisseur de cloud décide comment il est équilibré en charge.
- La création réelle de l'équilibreur de charge se produit de manière asynchrone.
- Chaque fois que vous souhaitez exposer un service au monde extérieur, vous devez créer un nouveau LoadBalancer et obtenir une adresse IP.

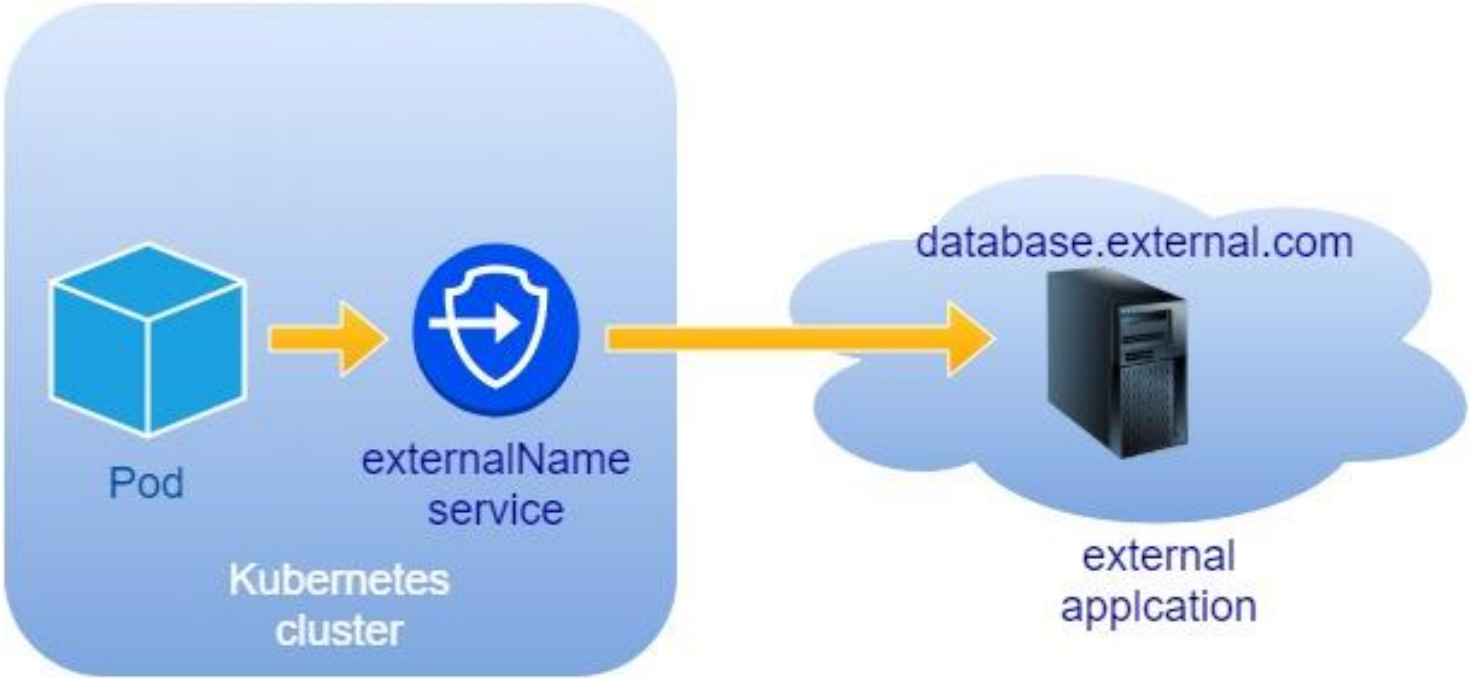
Cas d'utilisation

- Lorsque vous utilisez un fournisseur de cloud pour héberger votre cluster Kubernetes.



ExternalName

Pour pouvoir accéder à une application qui vit en dehors d'un cluster kubernetes , nous utilisons un service appelé externalName .

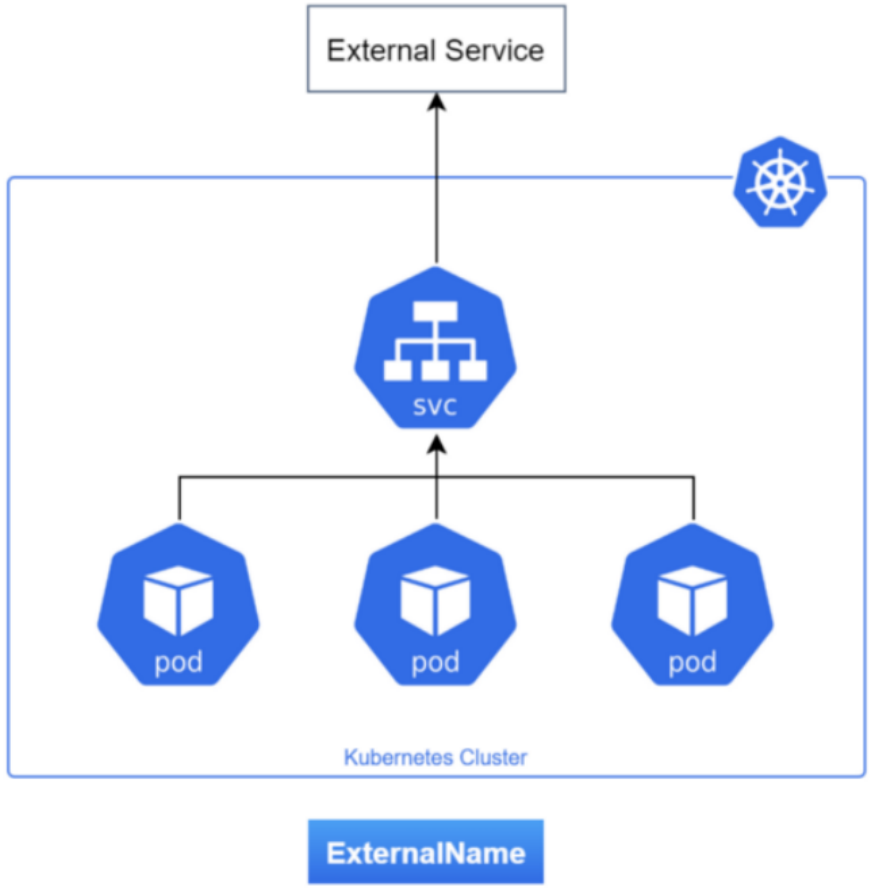


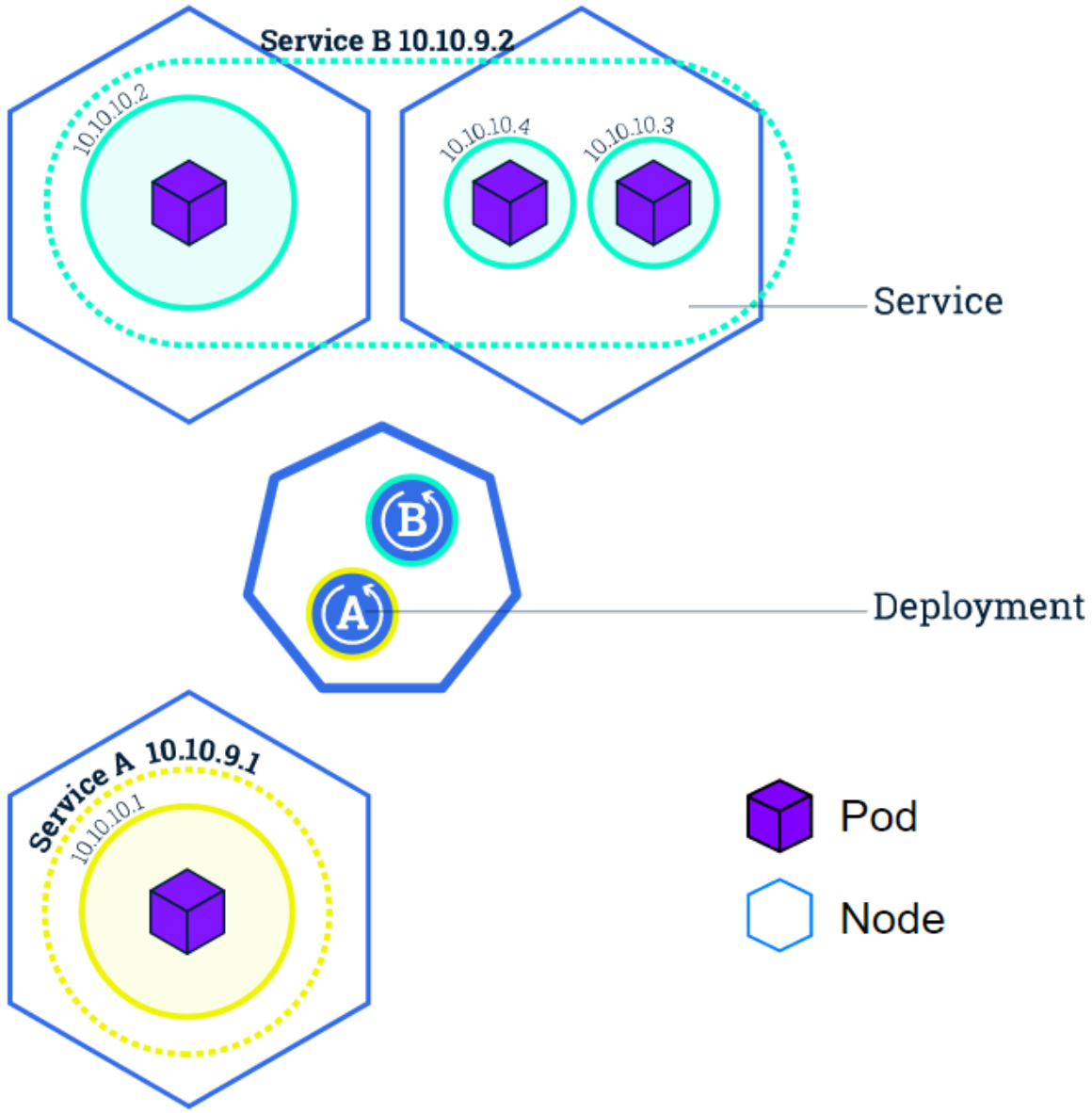
ExternalName

- Les services ExternalName mappent un service à un nom DNS.
- Vous spécifiez ces services avec le paramètre `spec.externalName`.
- Il mappe le service sur le contenu du champ externalName (par exemple foo.bar.example.com), en renvoyant un enregistrement CNAME avec sa valeur.
- Aucune procuration de quelque nature que ce soit n'est établie.

Cas d'utilisation

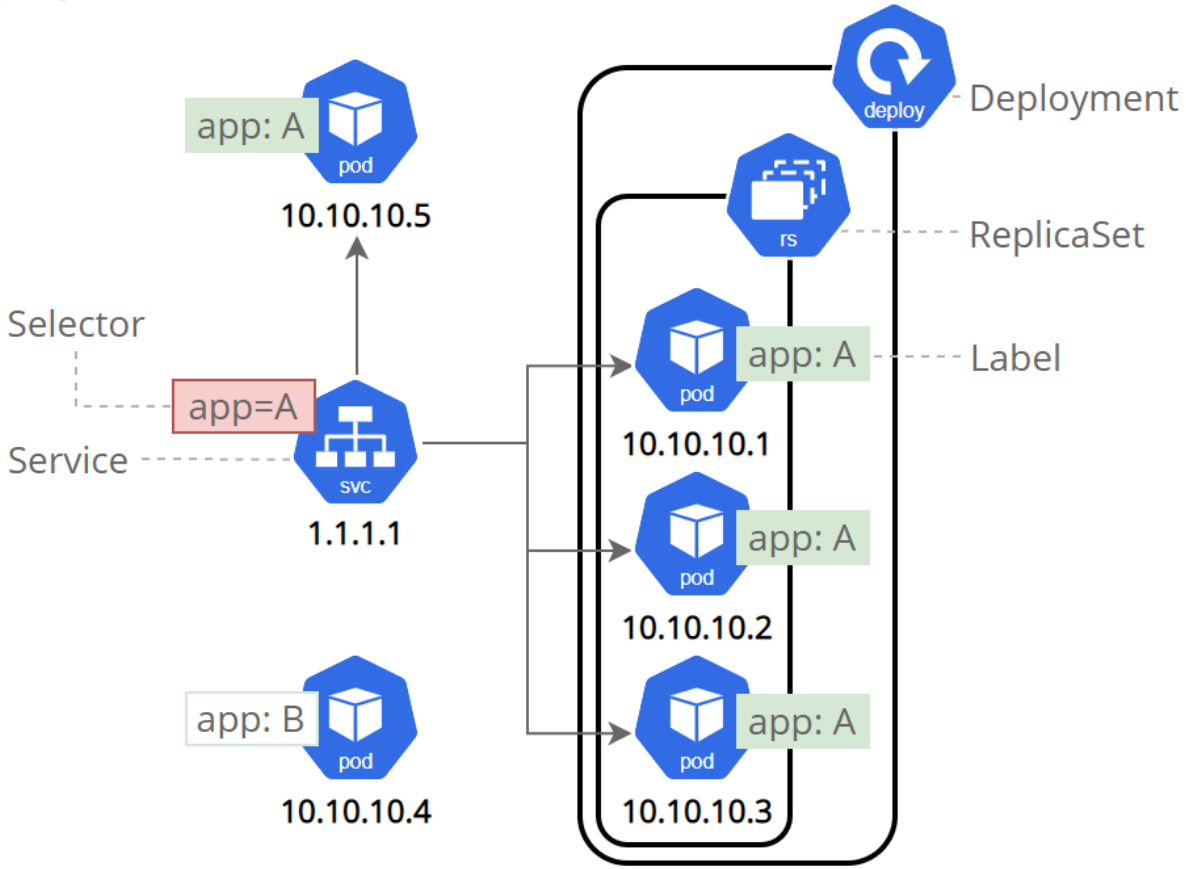
- Créer un service au sein de Kubernetes pour représenter une banque de données externe comme une base de données qui s'exécute en externe à Kubernetes.
- Utiliser ExternalName (en tant que service local) lorsque les pods d'un namespace communiquent avec un service dans un autre namespace.





Services & Labels
Ingress

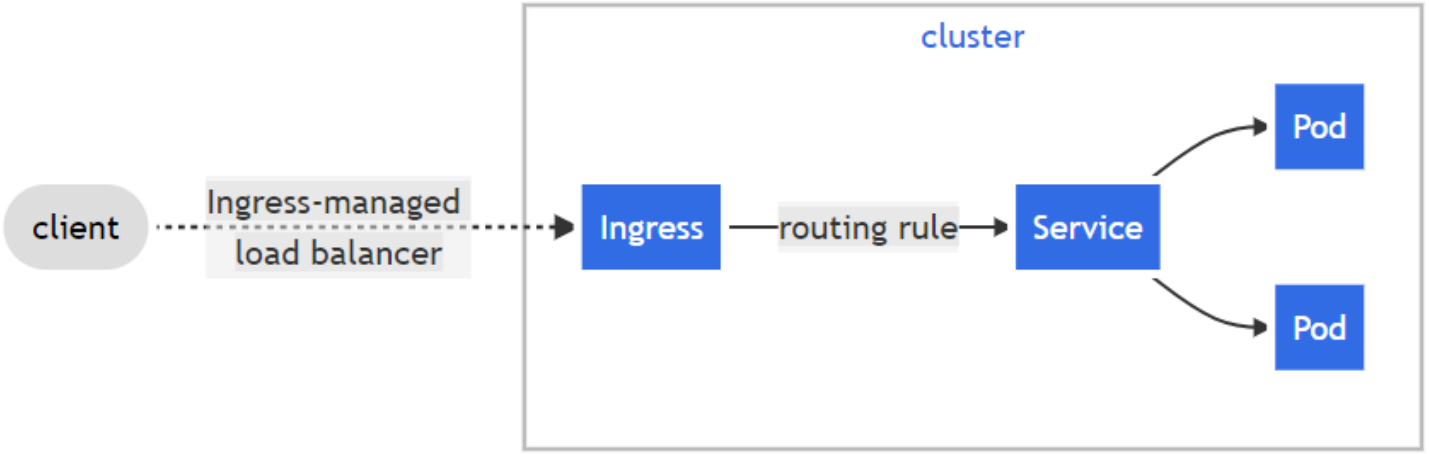
Services et Labels



Ingress

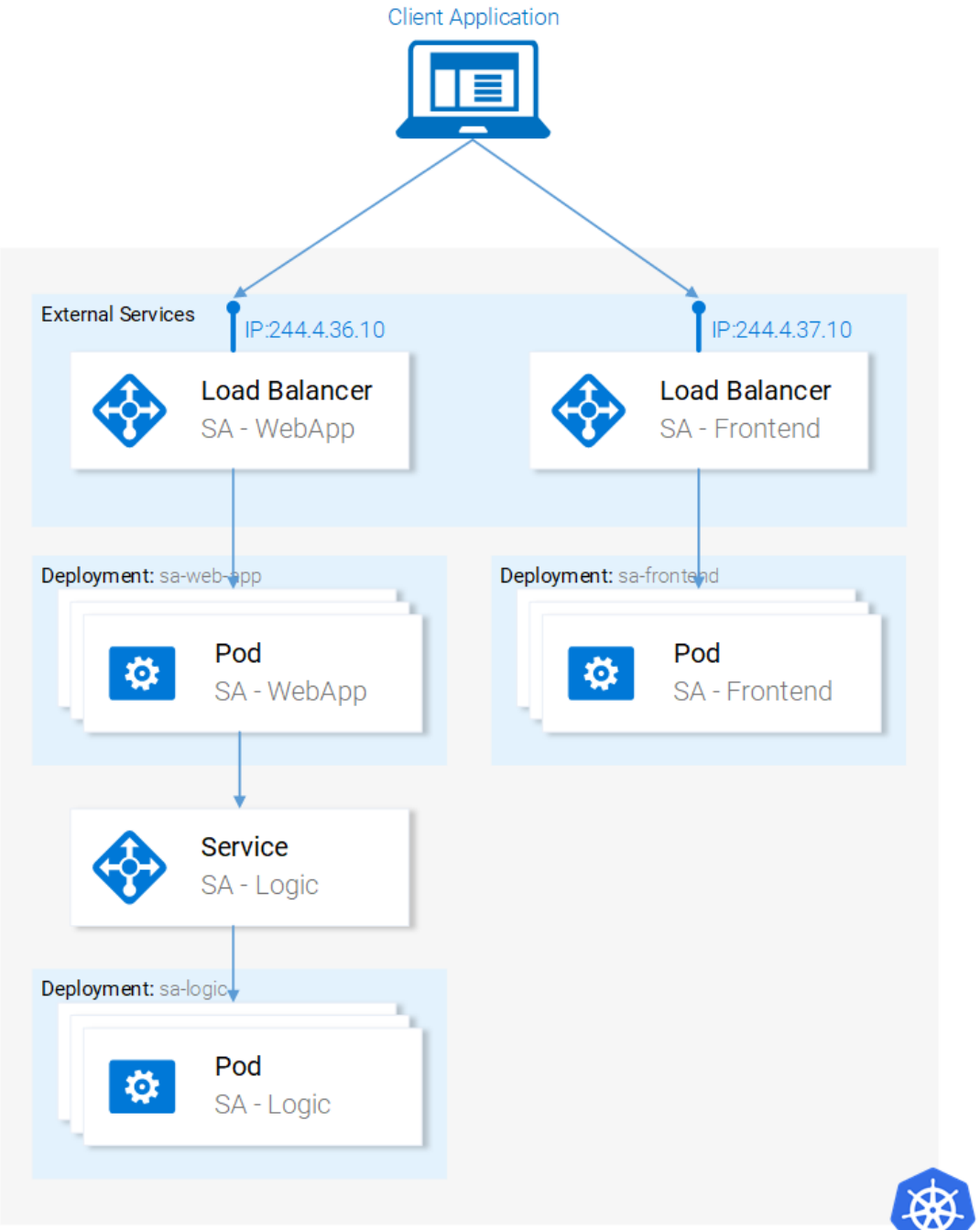
- Si l’application doit être accessible de l'extérieur de votre cluster, Kubernetes propose un composant appelé **Ingress**. Il se trouve au dessus du service et gère les requêtes externes avec le cryptage SSL.

- Kubernetes gère le scaling des pods pour gérer leur mise à jour par exemple avec des **déploiements**.



Elies Jebri : elies.jebri@gmail.com

En résumé



Elies Jebri : elies.jebri@gmail.com